

EXPERIMENT - 1

AIM:

To find and return the first palindromic string from a given array of strings. If no palindromic string exists, return an empty string.

```
def firstPalindrome(words):
```

```
    for word in words:
```

```
        if word == word[::-1]:
```

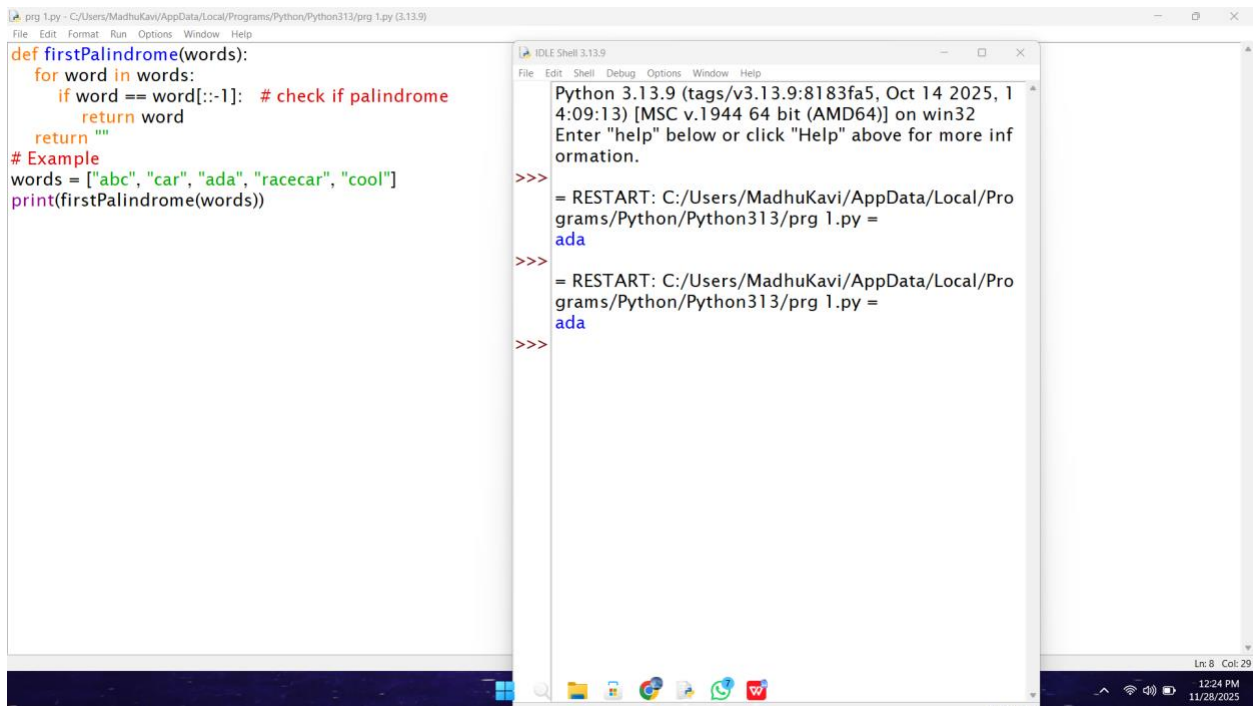
```
            return word
```

```
    return ""
```

```
words = ["abc", "car", "ada", "racecar", "cool"]
```

```
print(firstPalindrome(words))
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window displays the code for the `firstPalindrome` function and its execution. The right window shows the output of the program.

```
def firstPalindrome(words):
    for word in words:
        if word == word[::-1]: # check if palindrome
            return word
    return ""
# Example
words = ["abc", "car", "ada", "racecar", "cool"]
print(firstPalindrome(words))
```

The output window shows the following text:

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg 1.py =
ada
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg 1.py =
ada
>>>
```

The taskbar at the bottom shows the system clock as 12:24 PM on 11/28/2025.

EXPERIMENT-2

AIM:

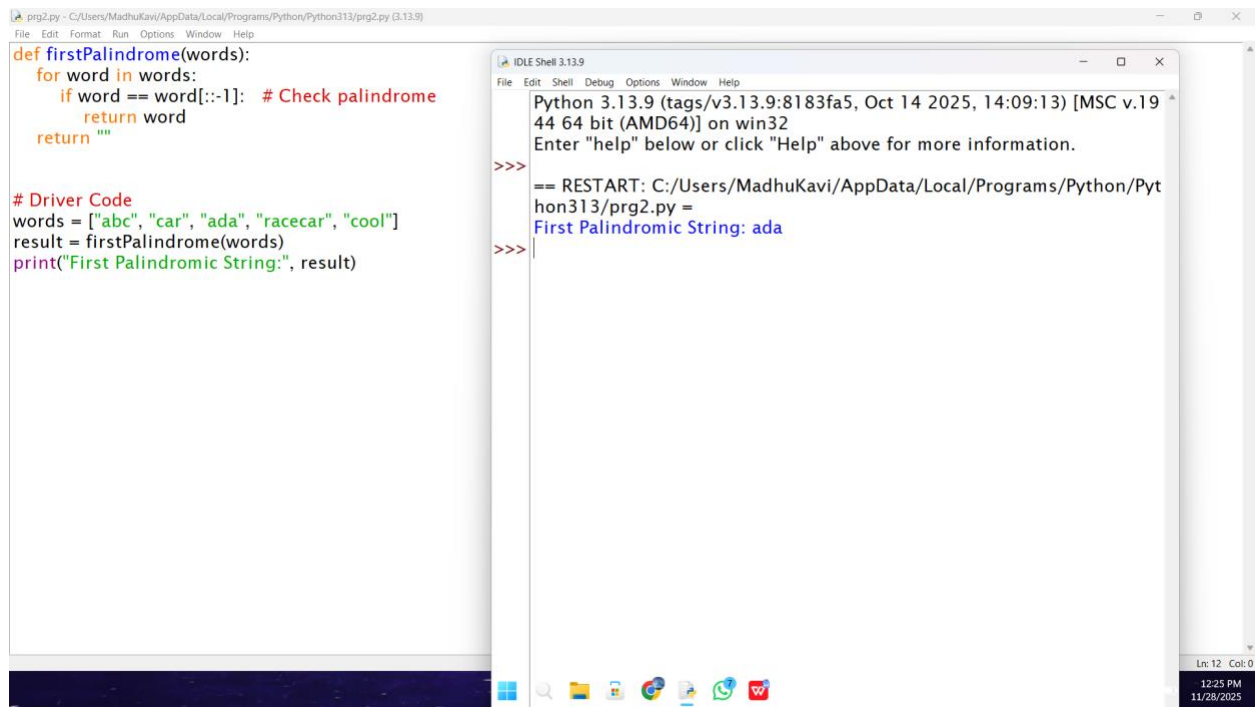
To calculate:

- answer1: the number of indices i such that nums1[i] exists in nums2
- answer2: the number of indices i such that nums2[i] exists in nums1
and return the result as [answer1, answer2].

PROGRAM:

```
def countCommon(nums1, nums2):  
    set1 = set(nums1)  
    set2 = set(nums2)  
    answer1 = 0  
    answer2 = 0  
    for x in nums1:  
        if x in set2:  
            answer1 += 1  
    for y in nums2:  
        if y in set1:  
            answer2 += 1  
    return [answer1, answer2]  
  
nums1 = [4, 3, 2, 3, 1]  
nums2 = [2, 2, 5, 2, 3, 6]  
result = countCommon(nums1, nums2)  
print("Output:", result)
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window displays a Python script for finding the first palindromic string in a list. The right window shows the execution output, indicating a restart and the result 'First Palindromic String: ada'.

```
prg2.py - C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg2.py (3.13.9)
File Edit Format Run Options Window Help

def firstPalindrome(words):
    for word in words:
        if word == word[::-1]: # Check palindrome
            return word
    return ""

# Driver Code
words = ["abc", "car", "ada", "racecar", "cool"]
result = firstPalindrome(words)
print("First Palindromic String:", result)
```

```
IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.19
44 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
== RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Pyt
hon313/prg2.py =
First Palindromic String: ada
>>>
```

Ln: 12 Col: 0
12:25 PM
11/28/2025

EXPERIMENT-3

AIM:

To find the sum of the squares of distinct counts of all subarrays of a given array.

PROGRAM:

```
def sumOfSquares(nums):
    n = len(nums)
    total = 0
    for i in range(n):
        s = set()
        for j in range(i, n):
            s.add(nums[j])
            total += len(s) ** 2
    return total

nums = [1, 2, 1]
print(sumOfSquares(nums))
```

OUTPUT:

```
prg3.py - C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg3.py (3.13.9)
File Edit Format Run Options Window Help

def countCommon(nums1, nums2):
    set1 = set(nums1)
    set2 = set(nums2)

    answer1 = 0
    answer2 = 0

    for x in nums1:
        if x in set2:
            answer1 += 1

    for y in nums2:
        if y in set1:
            answer2 += 1

    return [answer1, answer2]

# Driver Code
nums1 = [4, 3, 2, 3, 1]
nums2 = [2, 2, 5, 2, 3, 6]

result = countCommon(nums1, nums2)
print("Output:", result)
```

```
IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1914 64 b
it (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
== RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313
/prg3.py =
Output: [3, 4]
>>>
== RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313
/prg3.py =
Output: [3, 4]
>>>
```

EXPERIMENT-4

AIM:

To find the number of pairs (i, j) such that $\text{nums}[i] == \text{nums}[j]$ and $(i * j)$ is divisible by k.

PROGRAM:

```
def countPairs(nums, k):

    n = len(nums)

    count = 0

    for i in range(n):

        for j in range(i + 1, n):

            if nums[i] == nums[j] and (i * j) % k == 0:

                count += 1

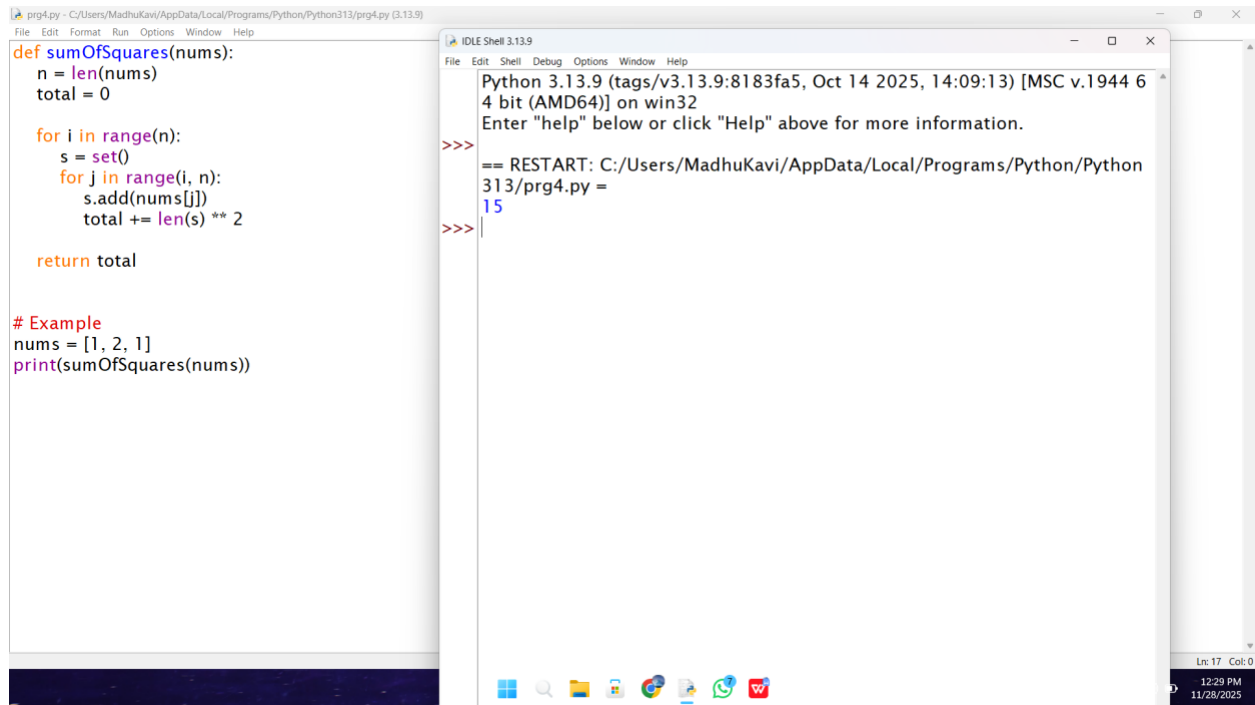
    return count

nums = [3,1,2,2,2,1,3]

k = 2

print(countPairs(nums, k))
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window, titled 'prg4.py', contains a function `sumOfSquares` that takes a list `nums` and returns the sum of the squares of its elements. It uses a nested loop and a set to calculate the sum. Below the function, an example is provided: `nums = [1, 2, 1]` and `print(sumOfSquares(nums))`. The right window, titled 'IDLE Shell 3.13.9', shows the execution of the script. It displays the Python version (3.13.9), the file path, and the output of the `print` statement, which is `15`. The status bar at the bottom right indicates 'Ln: 17 Col: 0' and the date '11/28/2025'.

```
prg4.py - C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg4.py (3.13.9)
File Edit Format Run Options Window Help

def sumOfSquares(nums):
    n = len(nums)
    total = 0

    for i in range(n):
        s = set()
        for j in range(i, n):
            s.add(nums[j])
            total += len(s) ** 2

    return total

# Example
nums = [1, 2, 1]
print(sumOfSquares(nums))

IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 6
4 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
== RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg4.py =
15
>>>

Ln: 17 Col: 0
12:29 PM
11/28/2025
```

EXPERIMENT-5

AIM:

To find the largest element in a given integer array with minimum time complexity.

PROGRAM:

```
def findMax(arr):

    max_val = arr[0]

    for x in arr:

        if x > max_val:

            max_val = x

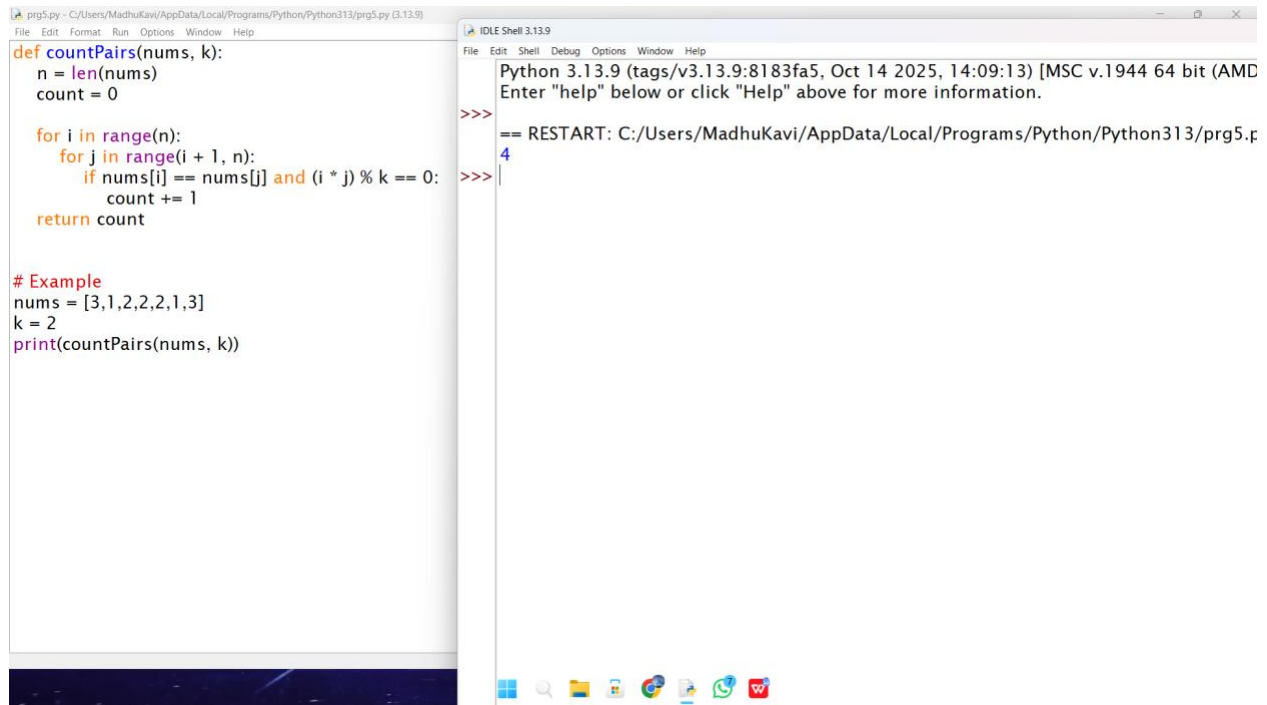
    return max_val

print(findMax([1, 2, 3, 4, 5]))

print(findMax([7, 7, 7, 7, 7]))

print(findMax([-10, 2, 3, -4, 5]))
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window, titled 'prg5.py', contains the following code:

```
def countPairs(nums, k):
    n = len(nums)
    count = 0

    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1
    return count

# Example
nums = [3,1,2,2,2,1,3]
k = 2
print(countPairs(nums, k))
```

The right window, titled 'IDLE Shell 3.13.9', shows the execution output:

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD
Enter "help" below or click "Help" above for more information.

>>>
== RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg5.p
4
>>>
```

EXPERIMENT-6

AIM:

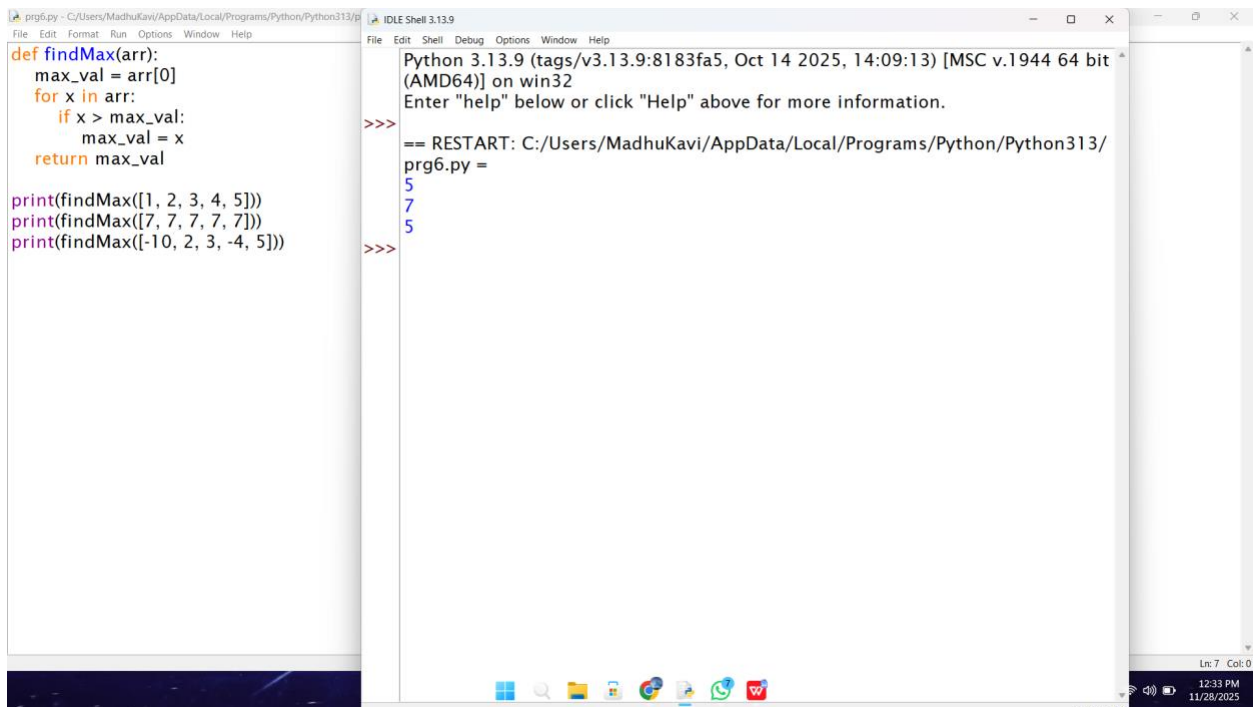
To sort a given list of numbers using an efficient sorting algorithm and find the maximum element, handling empty, single-element, and uniform lists.

PROGRAM:

```
def findMaxAfterSort(nums):
    if len(nums) == 0:
        return "List is empty"
    nums.sort()
    return nums[-1]

print(findMaxAfterSort([]))
print(findMaxAfterSort([5]))
print(findMaxAfterSort([3, 3, 3, 3, 3]))
```

OUTPUT:



The screenshot shows a Python IDLE Shell window with the following code and output:

```
def findMax(arr):  
    max_val = arr[0]  
    for x in arr:  
        if x > max_val:  
            max_val = x  
    return max_val  
  
print(findMax([1, 2, 3, 4, 5]))  
print(findMax([7, 7, 7, 7, 7]))  
print(findMax([-10, 2, 3, -4, 5]))
```

The output in the shell is:

```
>>>  
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit  
(AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
  
== RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/  
prg6.py =  
5  
7  
5  
>>>
```

EXPERIMENT-7

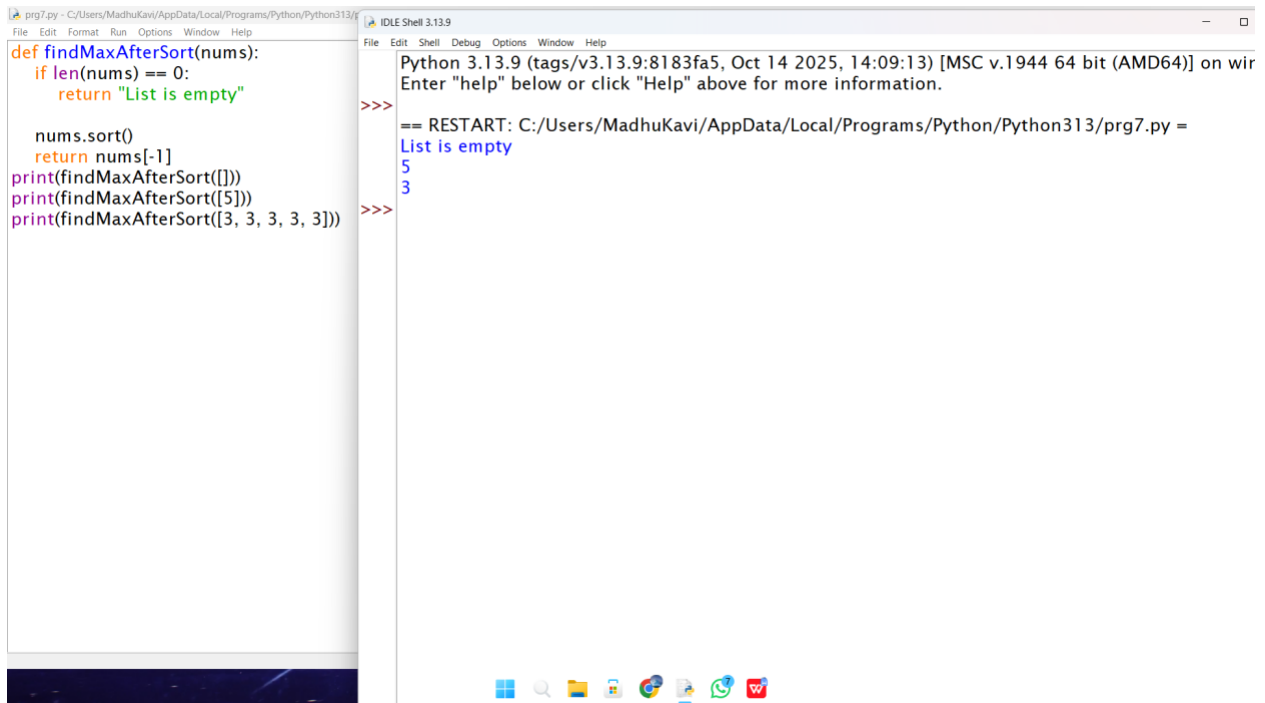
AIM:

To create a new list containing only unique elements from the given list of numbers.

PROGRAM:

```
def uniqueElements(nums):  
    seen = set()  
    unique = []  
    for x in nums:  
        if x not in seen:  
            seen.add(x)  
            unique.append(x)  
    return unique  
  
print(uniqueElements([3, 7, 3, 5, 2, 5, 9, 2]))  
print(uniqueElements([-1, 2, -1, 3, 2, -2]))  
print(uniqueElements([1000000, 999999, 1000000]))
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window, titled 'prg7.py', contains the following code:

```
def findMaxAfterSort(nums):  
    if len(nums) == 0:  
        return "List is empty"  
  
    nums.sort()  
    return nums[-1]  
print(findMaxAfterSort([]))  
print(findMaxAfterSort([5]))  
print(findMaxAfterSort([3, 3, 3, 3, 3]))
```

The right window, titled 'IDLE Shell 3.13.9', shows the output of the code:

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
  
>>> == RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg7.py =  
List is empty  
5  
3  
>>>
```

EXPERIMENT-8

AIM:

To sort an array of integers using the Bubble Sort technique and analyze its time complexity using Big-O notation.

PROGRAM:

```
def bubbleSort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        for j in range(n - 1 - i):
```

```
            if arr[j] > arr[j + 1]:
```

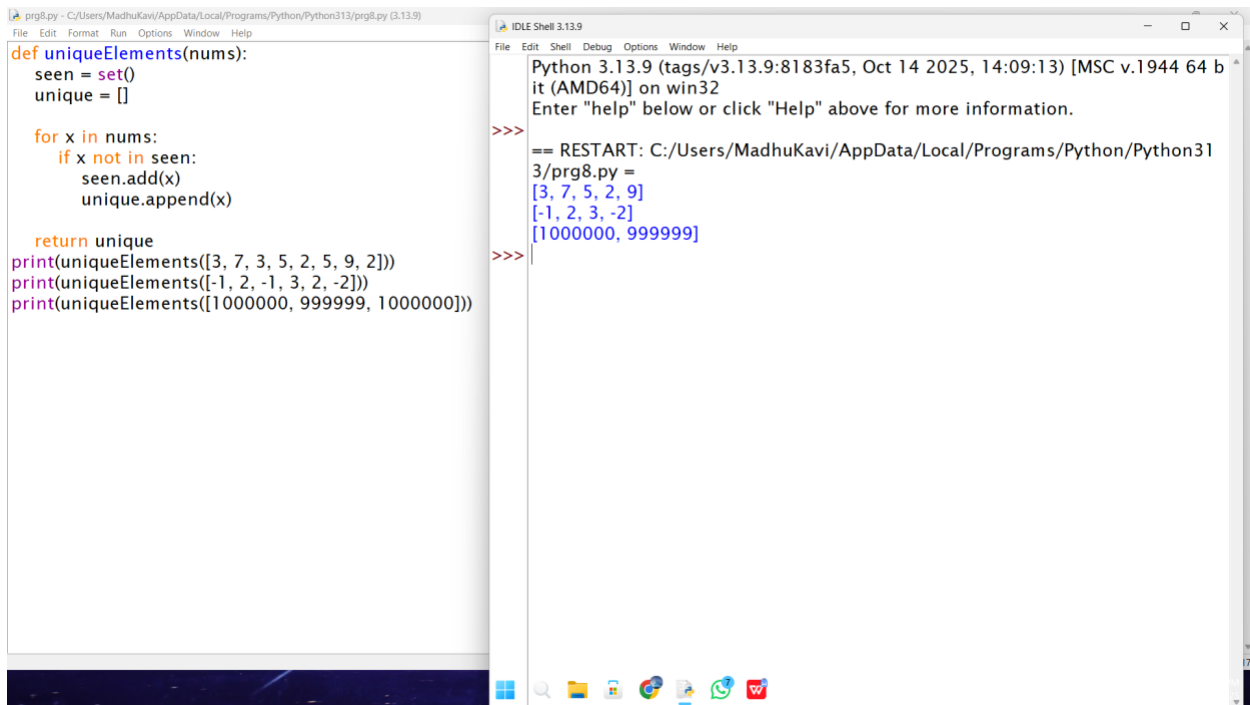
```
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
    return arr
```

```
nums = [5, 1, 4, 2, 8]
```

```
print("Sorted Array:", bubbleSort(nums))
```


OUTPUT:



The screenshot shows a Python IDE with two windows. The left window, titled 'prg8.py', contains the following code:

```
def uniqueElements(nums):  
    seen = set()  
    unique = []  
  
    for x in nums:  
        if x not in seen:  
            seen.add(x)  
            unique.append(x)  
  
    return unique  
print(uniqueElements([3, 7, 3, 5, 2, 5, 9, 2]))  
print(uniqueElements([-1, 2, -1, 3, 2, -2]))  
print(uniqueElements([1000000, 999999, 1000000]))
```

The right window, titled 'IDLE Shell 3.13.9', shows the output of the program:

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 b  
it (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
  
>>> == RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python31  
3/prg8.py =  
[3, 7, 5, 2, 9]  
[-1, 2, 3, -2]  
[1000000, 999999]  
>>>
```

EXPERIMENT-9

AIM:

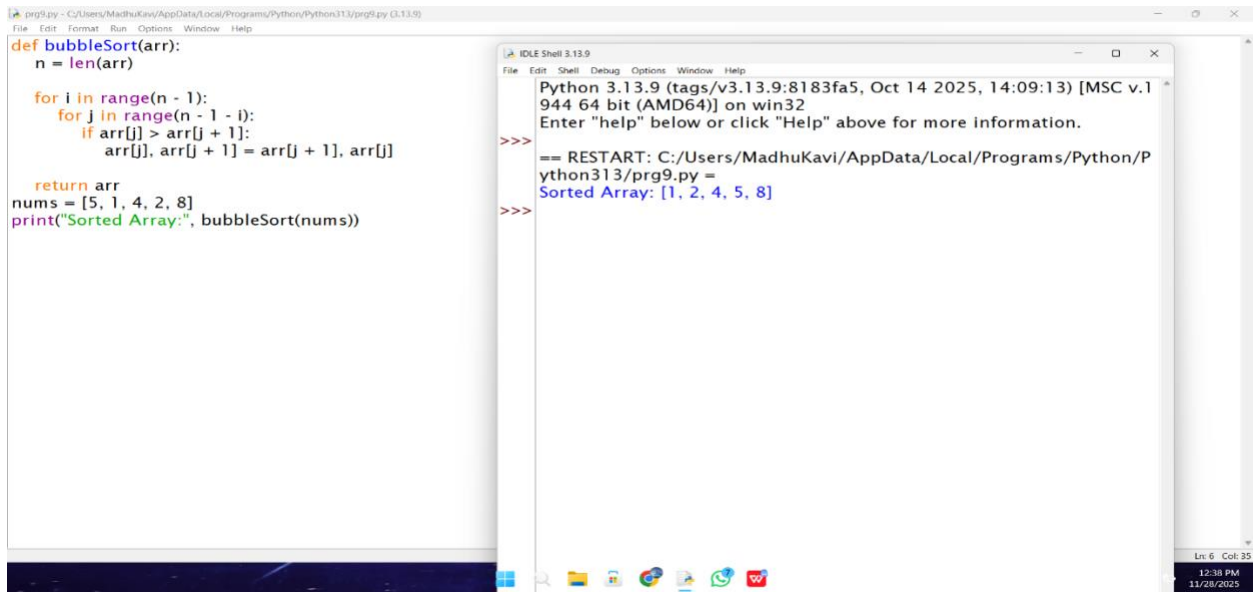
To check whether a given number KEY exists in a list using the Binary Search technique.

PROGRAM:

```
def binarySearch(arr, key):  
  
    arr.sort()  
  
    low = 0  
  
    high = len(arr) - 1  
  
    while low <= high:  
  
        mid = (low + high) // 2  
  
        if arr[mid] == key:  
  
            return mid + 1  
  
        elif arr[mid] < key:  
  
            low = mid + 1
```

```
        else:
            high = mid - 1
        return -1
X = [3, 4, 6, -9, 10, 8, 9, 30]
KEY = 10
pos = binarySearch(X, KEY)
if pos != -1:
    print("Element", KEY, "is found at position", pos)
else:
    print("Element", KEY, "is not found")
X = [3, 4, 6, -9, 10, 8, 9, 30]
KEY = 100
pos = binarySearch(X, KEY)
if pos != -1:
    print("Element", KEY, "is found at position", pos)
else:
    print("Element", KEY, "is not found")
```

OUTPUT:



EXPERIMENT-10

AIM:

To sort a given array of integers in ascending order without using any built-in functions, achieving $O(n \log n)$ time complexity with minimum space usage.

PROGRAM:

```
def partition(arr, low, high):
```

```
    pivot = arr[high]
```

```
    i = low - 1
```

```
    for j in range(low, high):
```

```
        if arr[j] < pivot:
```

```
            i += 1
```

```
            arr[i], arr[j] = arr[j], arr[i]
```

```
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
```

```
    return i + 1
```

```
def quickSort(arr, low, high):
```

```
    if low < high:
```

```
        pi = partition(arr, low, high)
```

```
        quickSort(arr, low, pi - 1)
```

```

    quickSort(arr, pi + 1, high)

nums = [5, 2, 3, 1]

quickSort(nums, 0, len(nums) - 1)

print("Sorted Array:", nums)

```

OUTPUT:

The screenshot shows a Python IDE with two windows. The left window displays a Python script with a `quickSort` function and test cases. The right window shows the output of the script, including a restart message and the results of the search for elements 10 and 100.

```

def quickSort(arr, key):
    arr.sort()
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid + 1
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

X = [3, 4, 6, -9, 10, 8, 9, 30]
KEY = 10
pos = binarySearch(X, KEY)
if pos != -1:
    print("Element", KEY, "is found at position", pos)
else:
    print("Element", KEY, "is not found")

X = [3, 4, 6, -9, 10, 8, 9, 30]
KEY = 100
pos = binarySearch(X, KEY)

if pos != -1:
    print("Element", KEY, "is found at position", pos)
else:
    print("Element", KEY, "is not found")

```

```

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg10.py =
Element 10 is found at position 7
Element 100 is not found
>>>

```

EXPERIMENT-11

AIM:

To find the number of ways a ball can move out of an $m \times n$ grid in exactly N steps starting from a given cell.

PROGRAM:

```

def findPaths(m, n, N, i, j):

    MOD = 10**9 + 7

    dp = [[0]*n for _ in range(m)]

    dp[i][j] = 1

    count = 0

    for _ in range(N):

        temp = [[0]*n for _ in range(m)]

        for r in range(m):

            for c in range(n):

                if dp[r][c] > 0:

```

```

        val = dp[r][c]

        if r == 0: count += val

        else: temp[r-1][c] += val

        if r == m-1: count += val

        else: temp[r+1][c] += val

        if c == 0: count += val

        else: temp[r][c-1] += val

        if c == n-1: count += val

        else: temp[r][c+1] += val

    dp = temp

    return count

print(findPaths(2, 2, 2, 0, 0))

print(findPaths(1, 3, 3, 0, 1))

```

OUTPUT:

The screenshot shows a Python IDE with two windows. The left window displays the source code for a quicksort algorithm. The right window shows the execution output, including a restart message and the sorted array [1, 2, 3, 5].

```

prg11.py - C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg11.py (3.13.9)
File Edit Format Run Options Window Help

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

def quickSort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quickSort(arr, low, pi - 1)
        quickSort(arr, pi + 1, high)

nums = [5, 2, 3, 1]
quickSort(nums, 0, len(nums) - 1)
print("Sorted Array:", nums)

```

```

IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg10.py =
Element 10 is found at position 7
Element 100 is not found

>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg11.py
Sorted Array: [1, 2, 3, 5]

>>>

```

19 Col: 0
2:42 PM
28/10/2025

EXPERIMENT-12

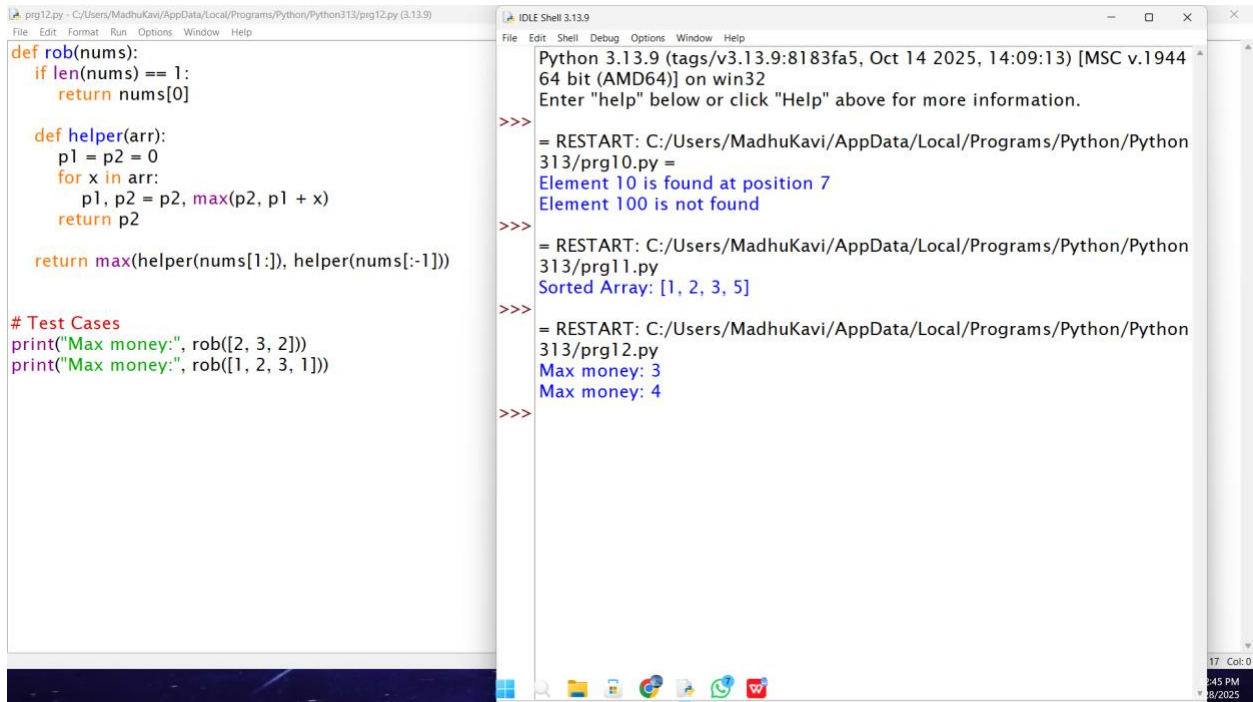
AIM:

To find the maximum amount of money that can be robbed from circularly arranged houses without robbing two adjacent houses on the same night.

PROGRAM:

```
def rob(nums):  
    if len(nums) == 1:  
        return nums[0]  
  
    def helper(arr):  
        p1 = p2 = 0  
        for x in arr:  
            p1, p2 = p2, max(p2, p1 + x)  
        return p2  
  
    return max(helper(nums[1:]), helper(nums[:-1]))  
  
print("Max money:", rob([2, 3, 2]))  
  
print("Max money:", rob([1, 2, 3, 1]))
```

OUTPUT:



The screenshot displays a Python IDE with two windows. The left window, titled 'prg12.py', contains the Python code for the 'rob' function and its test cases. The right window, titled 'IDLE Shell 3.13.9', shows the execution output. The code defines a 'rob' function that uses a 'helper' function to calculate the maximum money that can be robbed from a list of houses, considering the circular arrangement by checking two cases: starting from the first house and starting from the second house. The test cases are: `print("Max money:", rob([2, 3, 2]))` and `print("Max money:", rob([1, 2, 3, 1]))`. The output shows the first test case resulting in 'Max money: 3' and the second test case resulting in 'Max money: 4'.

```
def rob(nums):  
    if len(nums) == 1:  
        return nums[0]  
  
    def helper(arr):  
        p1 = p2 = 0  
        for x in arr:  
            p1, p2 = p2, max(p2, p1 + x)  
        return p2  
  
    return max(helper(nums[1:]), helper(nums[:-1]))  
  
# Test Cases  
print("Max money:", rob([2, 3, 2]))  
print("Max money:", rob([1, 2, 3, 1]))
```

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944  
64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg10.py =  
Element 10 is found at position 7  
Element 100 is not found  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg11.py  
Sorted Array: [1, 2, 3, 5]  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg12.py  
Max money: 3  
Max money: 4  
  
>>>
```

EXPERIMENT-13

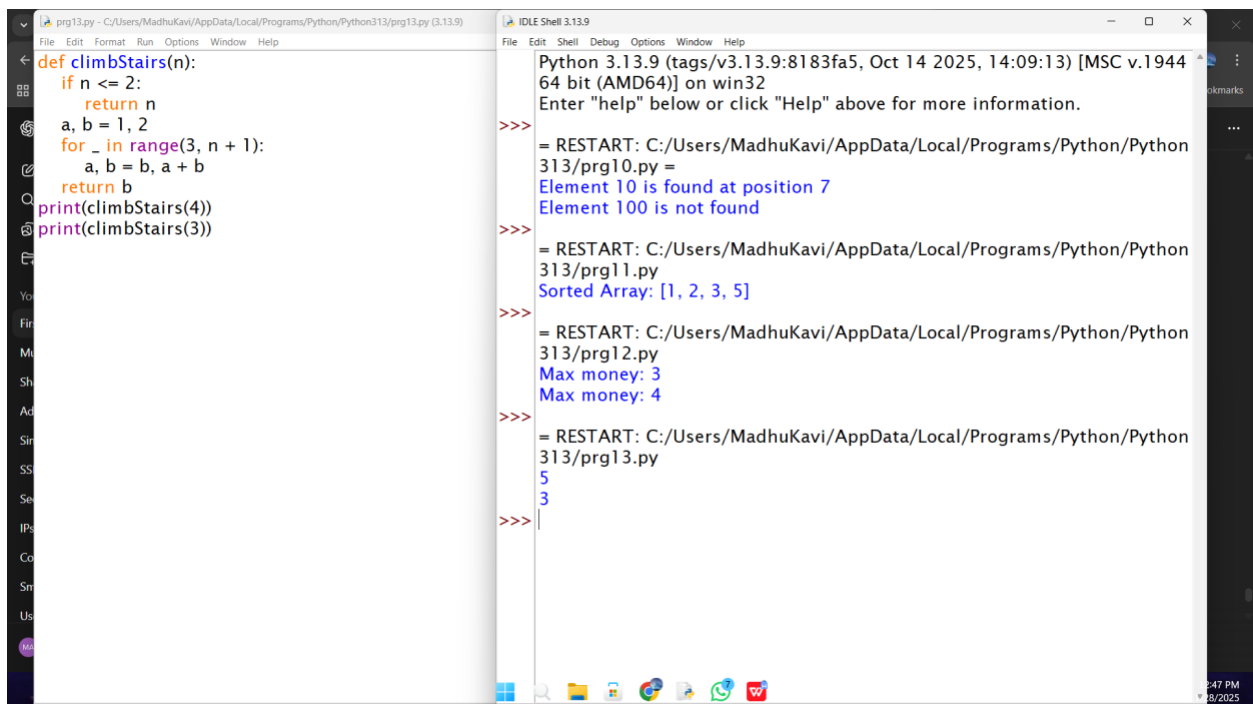
AIM:

To find the number of distinct ways to climb a staircase of n steps when only 1 or 2 steps can be taken at a time.

PROGRAM;

```
def climbStairs(n):  
    if n <= 2:  
        return n  
    a, b = 1, 2  
    for _ in range(3, n + 1):  
        a, b = b, a + b  
    return b  
  
print(climbStairs(4))  
print(climbStairs(3))
```

OUTPUT:



The screenshot shows a Python IDE with a file named 'prg13.py' open. The code in the file is as follows:

```
def climbStairs(n):  
    if n <= 2:  
        return n  
    a, b = 1, 2  
    for _ in range(3, n + 1):  
        a, b = b, a + b  
    return b  
  
print(climbStairs(4))  
print(climbStairs(3))
```

The IDE's output window shows the following execution results:

```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944  
64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg10.py =  
Element 10 is found at position 7  
Element 100 is not found  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg11.py  
Sorted Array: [1, 2, 3, 5]  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg12.py  
Max money: 3  
Max money: 4  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg13.py  
5  
3  
>>>
```

The output shows the results of the program for different values of n. For n=4, the output is 5. For n=3, the output is 3. The program is executed multiple times, showing the results for different values of n.

EXPERIMENT-14

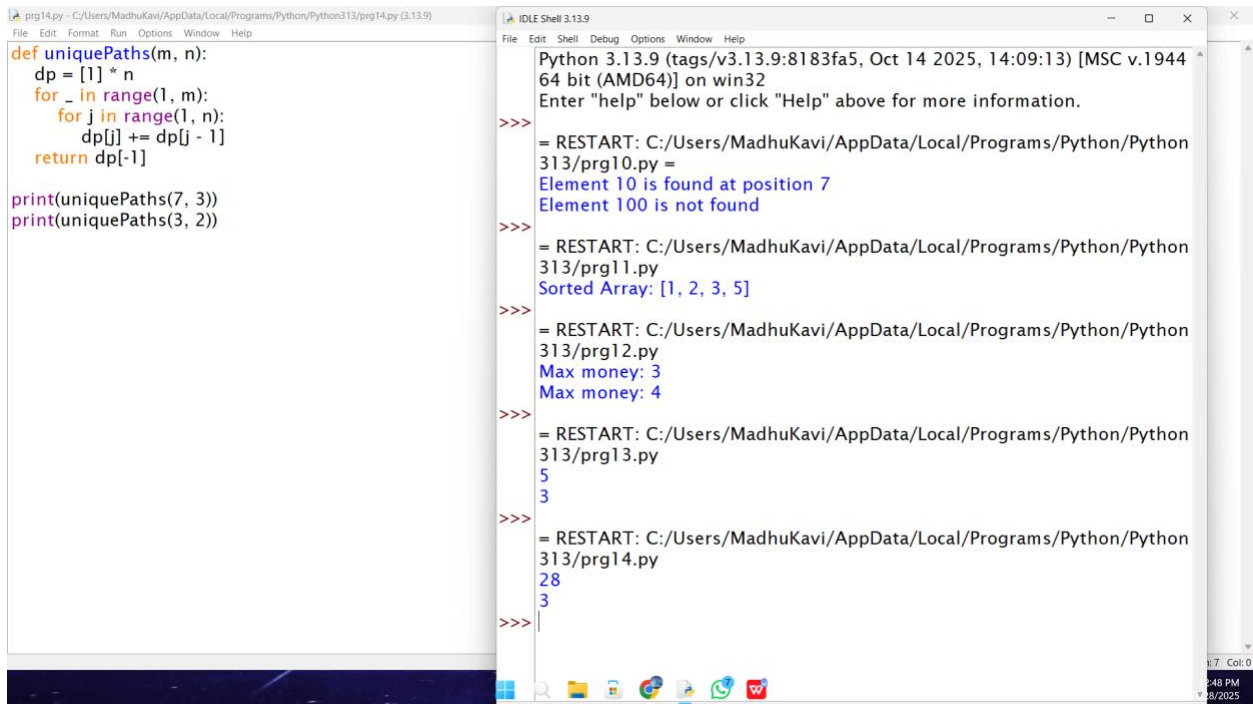
AIM:

To find the total number of unique paths for a robot to move from the top-left corner to the bottom-right corner of an $m \times n$ grid using only right and down moves.

PROGRAM:

```
def uniquePaths(m, n):  
    dp = [1] * n  
    for _ in range(1, m):  
        for j in range(1, n):  
            dp[j] += dp[j - 1]  
    return dp[-1]  
  
print(uniquePaths(7, 3))  
print(uniquePaths(3, 2))
```

OUTPUT:



```
prg14.py - C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg14.py (3.13.9)  
File Edit Format Run Options Window Help  
def uniquePaths(m, n):  
    dp = [1] * n  
    for _ in range(1, m):  
        for j in range(1, n):  
            dp[j] += dp[j - 1]  
    return dp[-1]  
  
print(uniquePaths(7, 3))  
print(uniquePaths(3, 2))  
  
IDLE Shell 3.13.9  
File Edit Shell Debug Options Window Help  
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944  
64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg10.py =  
Element 10 is found at position 7  
Element 100 is not found  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg11.py  
Sorted Array: [1, 2, 3, 5]  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg12.py  
Max money: 3  
Max money: 4  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg13.py  
5  
3  
  
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python  
313/prg14.py  
28  
3  
  
>>> |  
x: 7 Col: 0  
8/2025  
4:48 PM
```


EXPERIMENT-15

AIM:

To identify and return the intervals of all large groups (size ≥ 3) of consecutive identical characters in a given string.

PROGRAM:

def largeGroups(s):

```
    res = []
```

```
    i = 0
```

```
    n = len(s)
```

```
    while i < n:
```

```
        j = i
```

```
        while j < n and s[j] == s[i]:
```

```
            j += 1
```

```
        if j - i  $\geq$  3:
```

```
            res.append([i, j - 1])
```

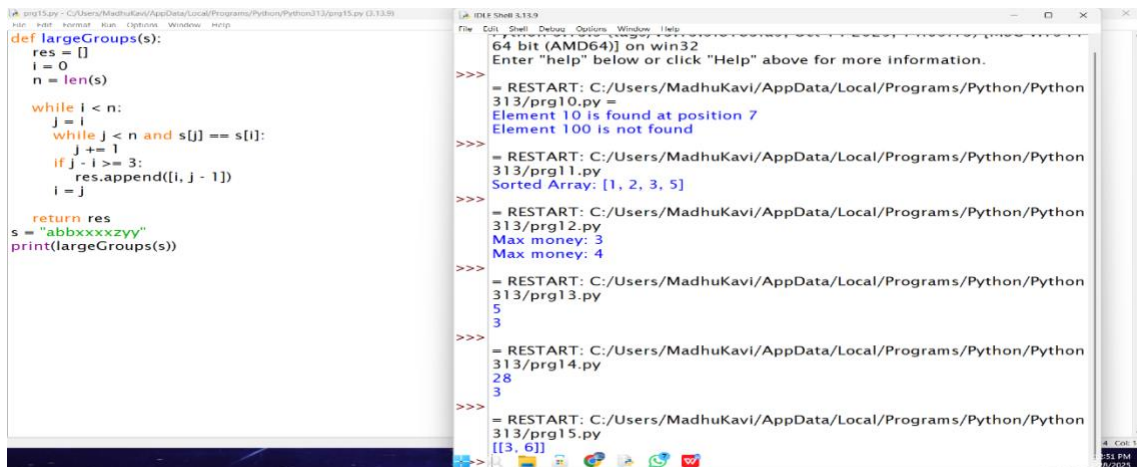
```
        i = j
```

```
    return res
```

```
s = "abbxxxxzzyy"
```

```
print(largeGroups(s))
```

OUTPUT:



The screenshot shows a Python IDE with two windows. The left window displays the code for the `largeGroups` function and its execution. The right window shows the output of the program, which includes the function definition, the string `s = "abbxxxxzzyy"`, and the printed result `[[3, 6]]`. The output also shows several restart messages for other files.

```
def largeGroups(s):
    res = []
    i = 0
    n = len(s)
    while i < n:
        j = i
        while j < n and s[j] == s[i]:
            j += 1
        if j - i  $\geq$  3:
            res.append([i, j - 1])
        i = j
    return res
s = "abbxxxxzzyy"
print(largeGroups(s))
```

```
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg10.py =
64 bit (AMD64) on win32
Enter "help" below or click "Help" above for more information.
>>> Element 10 is found at position 7
Element 100 is not found
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg11.py =
Sorted Array: [1, 2, 3, 5]
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg12.py =
Max money: 3
Max money: 4
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg13.py =
5
3
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg14.py =
28
3
>>> = RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg15.py =
[[3, 6]]
```

EXPERIMENT-16

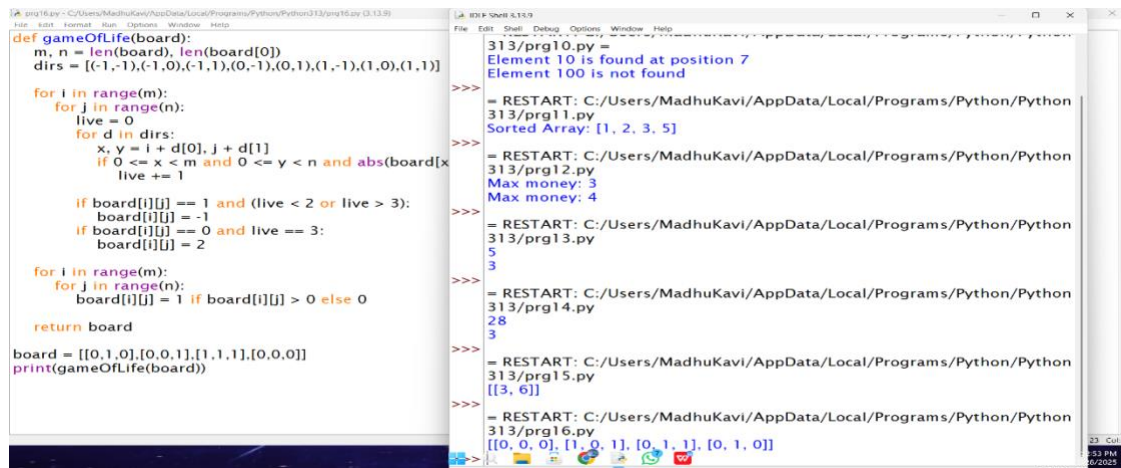
AIM:

To compute the next state of the Game of Life grid by applying the given rules to each cell simultaneously based on its live neighbors.

PROGRAM:

```
def gameOfLife(board):
    m, n = len(board), len(board[0])
    dirs = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
    for i in range(m):
        for j in range(n):
            live = 0
            for d in dirs:
                x, y = i + d[0], j + d[1]
                if 0 <= x < m and 0 <= y < n and abs(board[x][y]) == 1:
                    live +=1
            if board[i][j] == 1 and (live < 2 or live > 3):
                board[i][j] = -1
            if board[i][j] == 0 and live == 3:
                board[i][j] = 2
    for i in range(m):
        for j in range(n):
            board[i][j] = 1 if board[i][j] > 0 else 0
    return board
board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
print(gameOfLife(board))
```

OUTPUT:



```
def gameOfLife(board):
    m, n = len(board), len(board[0])
    dirs = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
    for i in range(m):
        for j in range(n):
            live = 0
            for d in dirs:
                x, y = i + d[0], j + d[1]
                if 0 <= x < m and 0 <= y < n and abs(board[x][y] - board[i][j]) < 2:
                    live += 1
            if board[i][j] == 1 and (live < 2 or live > 3):
                board[i][j] = 0
            if board[i][j] == 0 and live == 3:
                board[i][j] = 1
    for i in range(m):
        for j in range(n):
            board[i][j] = 1 if board[i][j] > 0 else 0
    return board
board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
print(gameOfLife(board))
```

```
313/prg10.py =
Element 10 is found at position 7
Element 100 is not found
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg11.py
Sorted Array: [1, 2, 3, 5]
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg12.py
Max money: 3
Max money: 4
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg13.py
5
3
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg14.py
28
3
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg15.py
[[3, 6]]
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg16.py
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
```

EXPERIMENT-17

AIM:

To determine the amount of champagne in any glass of a pyramid arrangement after pouring a given number of cups, following the rule that excess liquid splits equally to the two glasses below.

PROGRAM:

```
def champagneTower(poured, row, glass):
```

```
    dp = [[0]*(i+1) for i in range(row+1)]
```

```
    dp[0][0] = poured
```

```
    for i in range(row):
```

```
        for j in range(i+1):
```

```
            excess = max(0, dp[i][j] - 1)
```

```
            dp[i+1][j] += excess / 2
```

```
            dp[i+1][j+1] += excess / 2
```

```
    return min(1, dp[row][glass])
```

```
print(champagneTower(4, 2, 1))
```

OUTPUT:

```
prg17.py - C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python313/prg17.py (3.13.9)
File Edit Format Run Options Window Help

def champagneTower(poured, row, glass):
    dp = [[0]*(i+1) for i in range(row+1)]
    dp[0][0] = poured

    for i in range(row):
        for j in range(i+1):
            excess = max(0, dp[i][j] - 1)
            dp[i+1][j] += excess / 2
            dp[i+1][j+1] += excess / 2

    return min(1, dp[row][glass])
print(champagneTower(4, 2, 1))

IDLE Shell 3.13.9
File Edit Shell Debug Options Window Help

= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg11.py
Sorted Array: [1, 2, 3, 5]
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg12.py
Max money: 3
Max money: 4
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg13.py
5
3
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg14.py
28
3
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg15.py
[[3, 6]]
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg16.py
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
>>>
= RESTART: C:/Users/MadhuKavi/AppData/Local/Programs/Python/Python
313/prg17.py
0.5
1 Col: 33
2:56 PM
18/2/2025
```